GÜMÜŞHANE ÜNİVERSİTESİ
İKTİSADİ ve İDARİ BİLİMLER FAKÜLTESİ
YÖNETİM BİLİŞİM SİTEMLERİ BÖLÜMÜ

YBS 207 Algoritma ve Programlamaya Giriş

Fonksiyonlar (Functions)

Öğr. Gör. Dr. Baki GÖKGÖZ

Giriş

- **Python'da Fonksiyonlar:**
  - Fonksiyonlar, bir programı modüler ve yeniden kullanılabilir parçalara ayırmayı sağlar.
  - Modüler programlama, mantık parçalarını izole ederek karmaşık kod tabanlarının yönetimini kolaylaştırır.
  - Python'un fonksiyon yapısı, birçok diğer dile kıyasla daha güçlü ve özellik bakımından daha zengindir.
- **Fonksiyonların Önemi:**
  - Fonksiyonlar, programın okunabilirliğini ve sürdürülebilirliğini artırır.
  - Kodun yeniden kullanılmasını sağlar ve tekrarları azaltır.
  - Fonksiyonlar, büyük problemleri daha küçük alt problemlere ayırma (parçalama) yaklaşımını vurgular.
- **Diğer Dillerle Karşılaştırma:**
  - Fonksiyonlar, neredeyse tüm programlama dillerinde bulunur.
  - Python fonksiyonları; varsayılan argümanlar, esnek argüman listeleri ve birinci sınıf fonksiyonlar gibi, diğer dillerde sınırlı olan ya da hiç bulunmayan özellikleri destekler.

Fonksiyonlar

- **Fonksiyon Tanımı**
  - Fonksiyon, belirli bir görevi yerine getiren bir kod parçasıdır.
  - Fonksiyonlar, girdi parametreleri alır, işlemler gerçekleştirir ve çıktı (sonuç) döndürür.
- **Yerleşik (Built-in) Fonksiyonlar**
  - Python tarafından önceden tanımlanmış ve sağlanmış fonksiyonlardır.
  - Örnekler:
    - `print()` – Çıktıyı ekrana yazdırır.
    - `input()` – Kullanıcıdan girdi alır.
    - `range(start, stop, step)` – Bir sayı dizisi üretir.
    - `len()` – Yinelenebilir (iterable) nesnelerin uzunluğunu döndürür.
    - `int(), str()` – Veri tipi dönüştürme fonksiyonlarıdır.
    - `max()` – Bir iterable içindeki en büyük elemanı döndürür.

Fonksiyonlar

- **Fonksiyonların Özellikleri**
  - Belirli bir görevi yerine getirir.
  - Girdi olarak parametreler alır.
  - Çıktı / sonuç döndürür.
  - Yeniden kullanılabilirdir: bir kez yazılır, birçok kez kullanılır.
  - Modüler programlamayı ve ekip tabanlı geliştirmeyi destekler.
- **Kullanıcı Tanımlı Fonksiyonlar**
  - Fonksiyon oluşturma sözdizimi (syntax):

```
def function_name(parameters):
return result
```

  - Programcıların özel (custom) işlevler tanımlamasına olanak sağlar.
  - Büyük programların daha küçük ve yönetilebilir parçalara ayrılmasına yardımcı olur.
  - Fonksiyonlar, Python'da modül adı verilen dosyalar içinde gruplanabilir.

Fonksiyonlar

- **Fonksiyonların Avantajları**
  - Kodun yeniden kullanılması tekrarları azaltır.
  - Hata ayıklamayı ve bakım süreçlerini kolaylaştırır.
  - Ekip çalışmasını kolaylaştırır.
  - Programların okunabilirliğini ve yapısal düzenini geliştirir.
  - Uygulamaların kolayca genişletilmesine ve değiştirilmesine olanak tanır.

Fonksiyonlar

- **Python'da Fonksiyon Tanımı**
  - Bir fonksiyon tanımlamak için def anahtar kelimesi kullanılır.
  - Sözdizimi (Syntax):

```
def function_name(parameter_list):
return value
```

- **Fonksiyon adı, değişken adlandırma kurallarına uymalıdır:**
  - Python'un yerleşik `(built-in)` fonksiyon adları kullanılmamalıdır.
  - Program içinde benzersiz `(unique)` olmalıdır.
  - `()` içindeki parametre listesi, fonksiyonun ihtiyaç duyduğu girdileri belirtir
  - İki nokta üst üste `:` işareti, fonksiyon gövdesinin başladığını belirtir.
  - Fonksiyon gövdesi, tutarlı bir girintileme (indentation) ile yazılan ifadelerden oluşur.
  - Fonksiyondan bir `sonuç/çıktı` döndürmek için `return` ifadesi kullanılır.

Fonksiyonlar

- **Fonksiyon Başlığı (Header) veya İmzası (Signature)**
  - Fonksiyonun ilk satırı `(def volume(length, breadth, height):)` başlık ya da imza olarak adlandırılır.
  - Bu satır, fonksiyon adını ve parametrelerini içerir.
  - Diğer bazı dillerde bu yapı fonksiyon prototipi olarak da adlandırılır.
- **Parametreler ve Argümanlar**
  - Biçimsel Parametreler (Formal Parameters):
    - Fonksiyon tanımında listelenen değişkenlerdir
      - `(örnek: length, breadth, height)`
  - Gerçek Parametreler (Argümanlar / Actual Parameters):
    - Fonksiyon çağrılırken gönderilen değerler veya değişkenlerdir
      - `(örnek: volume(10, 5, 3))`
  - Parametre ve argüman terimleri bazen birbirinin yerine kullanılabilmektedir.

Fonksiyonlar

- **Fonksiyon Çağırma**
  - Bir fonksiyonu çalıştırmak için, fonksiyon adı yazılır ve ardından argümanların bulunduğu parantezler `()` kullanılır.
  - Fonksiyon, girdi parametrelerini işler ve bir değer döndürür.
  - Döndürülen değer, daha sonra kullanılmak üzere bir değişkende saklanabilir.
- **Örnek: Dikdörtgenler Prizmasının Hacmi**
  - Bir dikdörtgenler prizmasının :
    - **`hacmi = uzunluk × genişlik × yükseklik`**
  - Fonksiyon, üç parametresini çarparak hacmi hesaplar.

```
def volume(length, breadth, height):
        vol = length * breadth * height
return vol
v = volume(10, 5, 3) print(v)
# Output: 150
```

Fonksiyonlar

- **Konumsal (Positional) Argümanlar**
  - Değerler, parametrelere sıralarına göre aktarılır.
  - İlk argüman birinci parametreye, ikinci argüman ikinci parametreye karşılık gelir; bu şekilde devam eder.
  - Sıra değiştirildiğinde, değerler farklı parametrelere atanır.
  - Örnek: `volume(10, 5, 3)`
  - çağrısında **`10 → length, 5 → breadth, 3 → height`** olarak atanır.
- **Anahtar Kelimeli (Keyword) Argümanlar**
  - Argümanlar, parametre adları açıkça belirtilerek gönderilir.
  - Argümanların herhangi bir sırayla verilmesine olanak tanır.
  - Örnek: **`volume(length=10, breadth=5, height=3)`**

Fonksiyonlar

- **Örnek:**

```
def volume(length, breadth, height):
    print(length, breadth, height)
    vol = length * breadth * height
    return vol
v = volume(10, 5, 3)
print(v)
# Output: 10 5 3 \n 150
v = volume(5, 10, 3)
print(v)
# Output: 5 10 3 \n 150
v = volume(length=10, breadth=5, height=3)
print(v)
# Output: 10 5 3 \n 150
v = volume(height=3, length=10, breadth=5)
print(v)
# Output: 10 5 3 \n 150
v = volume(10, 5, height=3)
print(v)
# Output: 10 5 3 \n 150
```